PROGRAMAÇÃO PARA CIÊNCIA DE DADOS · 2026.1

NumPy

A biblioteca que faz contas com muitos números de uma vez — hoje, com 6.433 corridas de táxi de Nova York

AULA 03 • PYTHON

EM UMA FRASE

O que é

NUMPY

Uma biblioteca do Python que faz a conta em **milhares de números de uma vez**.

Python comum

você repete a conta num laço, número por número

NumPy

você escreve a conta **uma vez**, e ela vale para todos

TRÊS CORRIDAS DE TÁXI, TRÊS COISAS

Para que serve

```
import numpy as np

distancia = np.array([1.6, 0.8, 10.5])   # milhas rodadas
tarifa    = np.array([7.0, 5.0, 33.0])   # dólares cobrados
```

✖

Calcular

a mesma conta em todos os valores

```
distancia * 1.61
# milhas → km
```

📊

Resumir

média, máximo — uma palavra cada

```
tarifa.mean()
# 15.0 dólares
```

🔎

Filtrar

só os valores que interessam

```
tarifa[distancia > 10]
# [33.] — só a longa
```

TÁXIS DE NOVA YORK · MARÇO DE 2019

6.433 corridas

distance quantas milhas rodou	**fare** quanto deu o taxímetro	**tip** quanto foi de gorjeta	**payment** dinheiro ou cartão

É o mesmo dado da **aula 1** — e hoje ele responde perguntas sem nenhum laço.

O PONTO DE PARTIDA DE TUDO

Carregar

```
import numpy as np, pandas as pd

df = pd.read_csv("dados/taxis.csv")

distancia = df["distance"].to_numpy()
tarifa    = df["fare"].to_numpy()
gorjeta   = df["tip"].to_numpy()
```

Os **mesmos nomes** de antes — agora com as **6.433 corridas** do arquivo.



Na prática

O mesmo problema, dos dois jeitos

EXEMPLO 1 · MILHAS PARA QUILÔMETROS

Converter

Sem NumPy

```
km = []

for d in distancia:
    km.append(d * 1.60934)
```

Com NumPy

```
km = distancia * 1.60934
```

Uma linha. A conta vale para as 6.433 corridas de uma vez.

A MESMA CONTA · 10 MILHÕES DE VALORES

O tempo



60 vezes mais rápido. E quanto maior o dado, maior a diferença.

EXEMPLO 2 · QUANTO OS TÁXIS FATURARAM

Somar

Sem NumPy

```
total = 0

for t in tarifa:
    total = total + t
```

Com NumPy

```
total = tarifa.sum()

# 84 214.87 dólares
```

Ponto e o nome da conta. É assim que funcionam quase todos os resumos.

EXEMPLO 3 · DESCREVER A DISTÂNCIA DAS CORRIDAS

Resumir

```
distancia.mean()    # média
distancia.min()     # menor
distancia.max()     # maior
distancia.std()     # desvio padrão

np.median(distancia)  # mediana
```

MÉDIA
3,02
milhas

MEDIANA
1,64
milhas

MENOR
0
não saiu do lugar

MAIOR
36,7
milhas

A corrida **típica** tem 1,64 milha. A média é o dobro: poucas corridas longas puxam a conta.

EXEMPLO 4 · SÓ AS CORRIDAS LONGAS

Filtrar

Sem NumPy

```
longas = []

for d in distancia:
    if d > 10:
        longas.append(d)
```

Com NumPy

```
longas = distancia[distancia > 10]
```

Leia como está escrito: **distância onde a distância é maior que 10**.

EXEMPLO 5 · DUAS CONDIÇÕES AO MESMO TEMPO

Combinar

```
longa_com_gorjeta = (distancia > 10) & (gorjeta > 0)

longa_com_gorjeta.sum()    # 225  de 394 corridas longas
```

CORRIDAS LONGAS

394

acima de 10 milhas

COM GORJETA

225

57% delas

`&` é "e" · `|` é "ou". Cada condição entre **parênteses**.

EXEMPLO 6 · CRIAR UMA COLUNA NOVA

Rotular

Sem NumPy

```
tipo = []

for d in distancia:
    if d > 10:
        tipo.append("longa")
    else:
        tipo.append("curta")
```

Com NumPy

```
tipo = np.where(distancia > 10,
                "longa",
                "curta")
```

`np.where` é o **if/else** aplicado à lista inteira: teste, resposta para o sim, resposta para o não.

EXEMPLO 7 · O QUE O PASSAGEIRO PAGOU

Juntar duas

Sem NumPy

```
pago = []

for i in range(len(tarifa)):
    pago.append(
        tarifa[i] + gorjeta[i])
```

Com NumPy

```
pago = tarifa + gorjeta
```

As duas colunas são somadas **corrida por corrida**: a primeira com a primeira, a segunda com a segunda.

EXEMPLO 8 · QUANTO CUSTA CADA MILHA

Dividir

```
por_milha = tarifa / distancia

np.nanmedian(por_milha)
# 5.42 dólares por milha
```

Nasce uma medida nova
preço por milha não estava no arquivo — saiu de duas colunas

51 corridas têm distância **zero**. Por isso o `nanmedian`: ele ignora essas.

EXEMPLO 9 · QUANTO SE DÁ DE GORJETA

Porcentagem

```
pct = gorjeta / tarifa * 100

np.median(pct[gorjeta > 0])    # 26.5%  entre quem deu gorjeta
```

DERAM GORJETA

64%

das corridas

QUANTO DERAM

26,5%

da tarifa, na mediana

Repare no encaixe: `pct[gorjeta > 0]` usa a máscara de **uma** coluna para selecionar em **outra**.

EXEMPLO 10 · O DADO IMPOSSÍVEL

O dado sujo

```
(distancia == 0).sum()   # 51

limpa = np.where(
    distancia == 0,   # o teste
    np.nan,           # onde deu certo
    distancia)        # onde não deu

distancia.mean()    # 3.02
np.nanmean(limpa)   # 3.05
```

Zero aqui é mentira
o carro não saiu do lugar e mesmo assim cobrou tarifa

`np.nan` é o "não sei"
não é zero: é a marca de que **ali não tem valor**

`nanmean` = média pulando os `nan`
existem também `nanmax`, `nanmin`, `nansum`

EXEMPLO 11 · ACHAR O EXTREMO

A mais cara

```
tarifa.max()      # 150.0  o VALOR
tarifa.argmax()   # 5364   a LINHA dele

# a linha 5364 é a mesma corrida
# em qualquer coluna:
distancia[5364]   # 36.7 milhas

np.sort(tarifa)[-3:]   # ordena e
# [143.5 150. 150.]    # pega as 3 últimas
```

`arg…` devolve posição

`max` responde **quanto**; `argmax` responde **qual linha**

E a posição liga as colunas

todas têm a mesma ordem, então dá para ir ver a corrida inteira

Sempre visite o extremo

US$ 150 em 36,7 milhas: caro porque foi longe — não é erro

EXEMPLO 12 · TESTAR SEM ARQUIVO

Inventar dados

```
np.arange(0, 10, 2)      # [0 2 4 6 8]
np.zeros(5)              # [0. 0. 0. 0. 0.]

rng = np.random.default_rng(42)
rng.normal(13, 5, 1000)    # 1000 tarifas sorteadas
```

O **42** é a semente: com ela, o sorteio dá o **mesmo resultado** amanhã. Sem ela, seu relatório muda a cada execução.



Com gráficos

O NumPy faz a conta · o Matplotlib só desenha

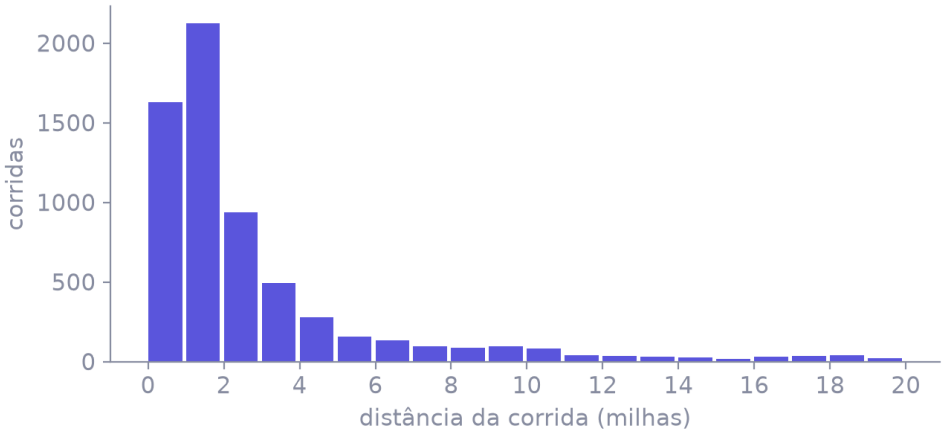
COMBINAÇÃO 1 · QUAL É A CORRIDA TÍPICA

Histograma

```
# bins = as bordas das faixas: 0, 1, 2 … 20
# (o 21 só marca o fim e não vira faixa)
alturas, faixas = np.histogram(distancia, bins=np.arange(0, 21, 1))

# alturas = quantas corridas caem em cada faixa
# faixas  = as bordas — sempre uma a mais que as barras

plt.bar(faixas[:-1], alturas)      # daí o [:-1]
```



O **NumPy conta** quantas corridas caem em cada faixa; o gráfico só desenha as barras. **2.125 corridas** ficaram entre 1 e 2 milhas.

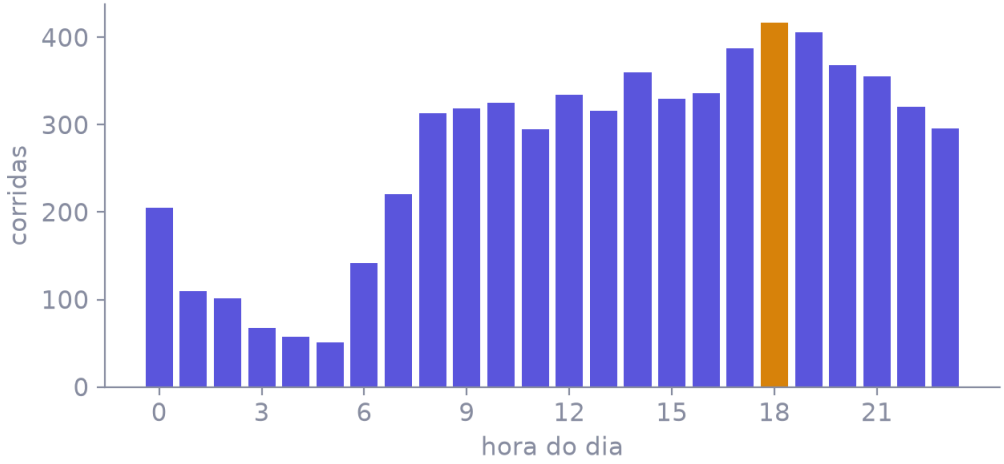
COMBINAÇÃO 2 · QUANDO AS PESSOAS PEGAM TÁXI

Por hora

```
hora = df["pickup"].dt.hour.to_numpy()     # 0 a 23

# bincount = "contagem por caixa": conta quantas vezes
# cada número aparece — a posição 18 guarda as das 18h
por_hora = np.bincount(hora)

plt.bar(np.arange(24), por_hora)
```



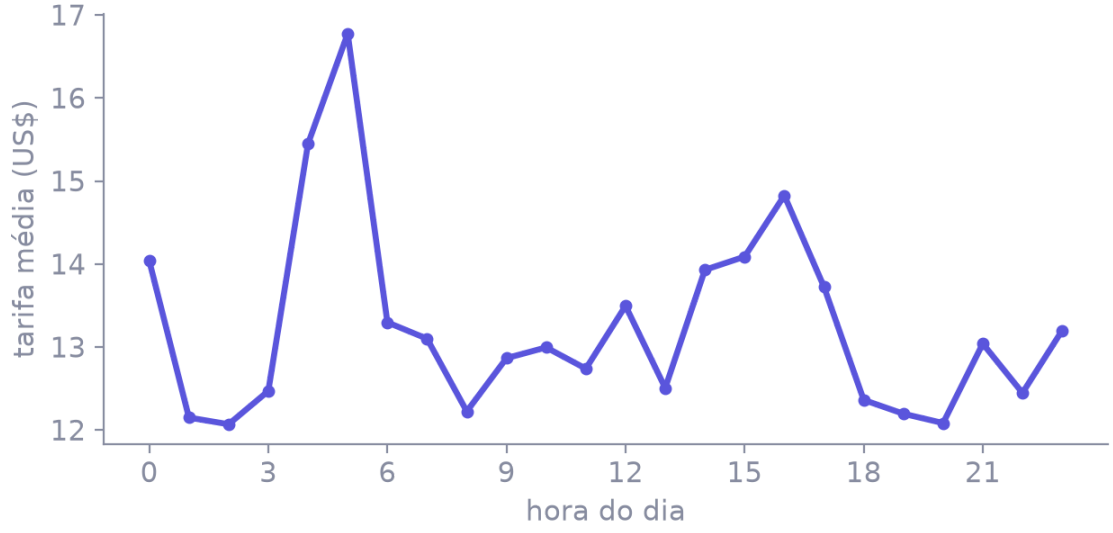
Pico às **18h** (417 corridas) e fundo do poço às **5h** (51). `np.bincount` conta as repetições de cada número inteiro.

COMBINAÇÃO 3 · O HORÁRIO MAIS CARO

Média por hora

```
# hora == h marca as corridas daquela hora;
# .mean() tira a média só delas
media = np.array([tarifa[hora == h].mean() for h in range(24)])

plt.plot(np.arange(24), media)     # x: 0 a 23 · y: a média
```



A tarifa média é maior às **5h da manhã** (US$ 16,77) e menor às **2h** (US$ 12,07). Pergunta para a turma: por quê?

AS TRÊS COMBINAÇÕES SÃO A MESMA COISA

O padrão

A coluna 1
6.433 números

›

NumPy 2
filtra, conta, resume

›

O gráfico 3
recebe pronto e desenha

O gráfico não faz conta. Se o desenho sair errado, o erro está no meio.

ONDE ISTO ENTRA NO DIA A DIA

No trabalho

- **Fechar o mês** — somar e tirar média da coluna inteira: `tarifa.sum()`
- **Responder uma pergunta** — recortar e medir: `tarifa[distancia > 10].mean()`
- **Criar coluna nova** — a regra vale para todo mundo: `np.where(distancia > 10, "longa", "curta")`
- **Regra por faixa** — comissão, alíquota, frete por peso: `np.select([...], [...])`
- **Achar o que está errado** — auditoria e fraude: `tarifa / distancia > 20`
- **Preparar o gráfico** — o NumPy conta, o gráfico só desenha: `np.histogram(distancia)`

```
# o relatório de segunda-feira, em quatro linhas
faturamento = tarifa.sum()                    # 84 214.87 dólares
ticket      = tarifa.mean()                   # 13.09 por corrida
longas      = (distancia > 10).mean() * 100   # 6.1 % do total
suspeitas   = (tarifa / distancia > 20).sum()  # 34 para investigar
```